

INTELLIGENT TRAFFIC MANAGEMENT SYSTEM

ITMS

Application & User Documentation

-49%Emergency Travel
Time**0.911**ML R² (30s)**132**

Tests Passing

7/7

Phases Complete

Team: Bitcoders**Event:** SISTec Innovation Hackathon 2026**Problem Statement:** IS-8**Date:** September 27, 2026

Documentation generated from source-code analysis
All metrics measured from actual SUMO simulation runs — nothing fabricated



ITMS — Application & User Documentation

Intelligent Traffic Management System *Command Center Operator Guide*

Application	ITMS Command Center
Framework	Next.js 15.5.6 + React 19
Team	Bitcoders
Event	SISTec Innovation Hackathon 2026

Table of Contents

- [Application Overview](#)
- [Accessing the Application](#)
- [Navigation Structure](#)
- [Command Center \(Dashboard\)](#)
- [Traffic Monitoring](#)
- [Emergency Management](#)
- [Signal Management](#)
- [Green Corridors](#)
- [SUMO Simulator](#)
- [Scenario Builder](#)
- [Analytics](#)
- [AI Decision Trace](#)
- [System Settings](#)
- [AI Copilot](#)
- [Roadside Devices](#)

- [Common Workflows](#)
- [Status Indicators](#)
- [Known Limitations](#)

1. Application Overview

The **ITMS Command Center** is the operator-facing mission control interface for the Intelligent Traffic Management System. It provides:

- **Live Map:** A real-time SVG visualization of the SUMO traffic network, showing vehicle positions, signal states, congestion levels, and emergency routes
- **Emergency Dispatch:** Controls to create and monitor emergency vehicle incidents
- **Green Corridor Control:** Real-time monitoring of the Predictive Green Corridor rolling wave
- **Traffic Analytics:** Performance comparison charts (Baseline vs ITMS)
- **AI Transparency:** Decision audit logs for every A* route, ETA, and safety constraint check
- **AI Copilot:** Natural language Q&A interface powered by Vultr Serverless Inference

The application is built with **Next.js 15 App Router** and uses **WebSocket** for continuous real-time data updates at approximately 5 Hz.

2. Accessing the Application

URLs

Service	URL	Status
Frontend (dev)	<code>http://localhost:3001</code>	<code>npm run dev:web</code>
Backend API	<code>http://localhost:3000</code>	<code>npm run dev:api</code>
WebSocket	<code>ws://localhost:3000/ws</code>	Auto-connected by frontend

ML Service

`http://localhost:8100`

```
cd services/prediction &&
uvicorn inference.service:app
--port 8100
```

Prerequisites

For full functionality:

- PostgreSQL (port 5433) must be running
- SUMO (v1.27.1) must be installed and in PATH
- Backend API must be started
- SUMO network must be built (`npm run build:network`)

Without the backend, pages will display a **disconnected banner** but all UI components remain visible.

3. Navigation Structure

The application uses a **collapsible sidebar navigation** (AppShell component). All pages are accessible from the sidebar.

Pages

Page	Route	Icon	Purpose
Command Center	<code>/</code>		Main dashboard / live map
Traffic Monitor	<code>/traffic</code>		Live road segment telemetry
Emergency Operations	<code>/emergencies</code>		Dispatch and monitor emergencies
Signal Control	<code>/signals</code>		Traffic signal states
Corridors	<code>/corridors</code>		Green corridor status
Simulator	<code>/simulator</code>		SUMO simulation controls
Scenarios	<code>/scenarios</code>		Comparison scenario builder
Analytics	<code>/analytics</code>		Performance metrics

AI Decision Trace	/decisions		AI routing/corridor audit log
Settings	/settings		System health status
AI Copilot	/ai		LLM-powered Q&A
Roadside Devices	/roadside-devices		IoT CRPD display status

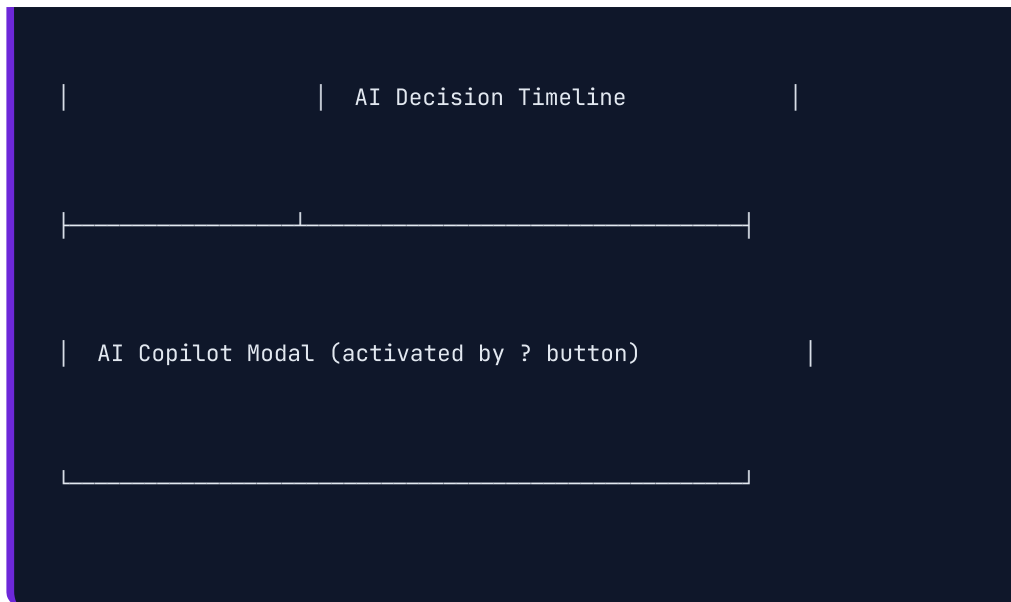
4. Command Center

Route: [/](#)

The main overview page. Provides a comprehensive single-pane view of the entire system.

Layout





Live Simulation Map Features

- **Junctions:** Displayed as colored circles. Traffic-light junctions are larger with ring animations.
- **Road Segments:** Lines colored by congestion: ● LOW → ● MEDIUM → ● HIGH → ● CRITICAL
- **Emergency Vehicles:** Animated pulsing indicator showing live GPS position from TraCI
- **Emergency Route:** Highlighted path from origin to destination
- **Green Corridor:** Highlighted intersections with active green overrides
- **Hover tooltips:** Junction name + signal state + queue length on hover
- **Zoom & Pan:** Via keyboard (arrow keys, + / -) or pinch gesture

Metric Cards

Four cards displayed in header:

- **Sim Time:** Formatted simulation clock
- **Active Emergencies:** Count with link to /emergencies
- **Active Corridors:** Count with link to /corridors
- **System Status:** Database / TraCI / ML health chips

AI Copilot Access

Click the **floating ? button** (bottom-right) to open the AI Copilot modal. Context is automatically injected: current traffic state, active emergencies, and which decision event you last hovered.

5. Traffic Monitoring

Route: </traffic>

What You See

Panel	Content
Summary Strip	Total vehicles, network-average speed, congestion distribution
Segment Table	Every road segment with: name, current count, speed (m/s), queue, occupancy, congestion badge
Junction Grid	Each junction with controlled/uncontrolled status and current queue
Heatmap	Color-coded congestion overlay (refreshes at 5 Hz)

Congestion Color Key

Color	Level	Occupancy	Speed Ratio	Queue
 Green	LOW	< 15%	> 60%	< 2
 Yellow	MEDIUM	< 30%	> 35%	< 6
 Orange	HIGH	< 45%	> 15%	< 12
 Red	CRITICAL	≥ 45%	≤ 15%	≥ 12

6. Emergency Management

Route: </emergencies>

Dispatching an Emergency

- Click **+ New Emergency** button (top-right)
- Fill in the dispatch form:
 - **Type:** Ambulance / Fire Engine / Police
 - **Priority:** Critical / High / Normal
 - **Origin Junction:** Select from live junction list
 - **Destination Junction:** Select from live junction list

- Click **Dispatch**
- The system will:
- Compute an A* route
- Persist to database
- Spawn the vehicle in SUMO via TraCI
- Automatically create a Green Corridor

Emergency Detail View

Each emergency card shows:

- **Status badge:** CREATED → ACTIVE → ARRIVED
- **Vehicle type** and icon
- **Origin** → **Destination** with junction IDs
- **Current speed** (m/s) from TraCI
- **Estimated ETA** (seconds remaining)
- **Route progress:** Number of junctions passed / total
- **Corridor status:** ACTIVE / PENDING / COMPLETED

Emergency Timeline

The right panel shows a chronological log:

- Emergency created
- Route computed (A* cost in seconds)
- Corridor activated
- Each corridor junction APPLIED / PASSED / SKIPPED
- Route switch events (if dynamic rerouting occurred)
- Vehicle arrival

7. Signal Management

Route: `/signals`

What You See

- **Grid of Signal Cards:** One card per traffic-light junction
- Each card shows:
- Junction ID + name

- Current phase index and state string (e.g., `GGrrGGrr`)
- Time until next switch (countdown)
- Queue length at this junction
- Congestion level badge

Manual Override

Click a signal card to open the **Signal Override Modal**:

- Select a phase index
- Enter a duration override (seconds)
- Click **Apply**
- The system calls `POST /api/signals/:id/state`

Note: Manual overrides are temporary — they last until the simulation program cycle completes or the corridor restores the normal program.

8. Green Corridors

Route: `/corridors`

What You See

- **Active Corridors:** Each corridor has:
 - Status: PLANNING → VALIDATING → ACTIVE → COMPLETED / CANCELLED
 - Emergency vehicle reference (type + ID)
 - Origin → Destination junctions
 - Junction count (how many signals are in the corridor)
 - Planned / activated timestamp
- **Corridor Chain:** Visual rail showing each junction in the corridor:
 - PENDING (●) — Waiting for vehicle to approach
 - APPLIED (♥) — Green signal currently active
 - PASSED (○) — Vehicle has passed through
 - SKIPPED (●) — Safety constraint rejected this junction
 - NOOP (○) — Normal green already covered ETA

Safety Constraint Annotations

If a junction was SKIPPED, the corridor card shows the rejection reason, e.g.:

- `"red_extension_limit"` — cross traffic would be held too long
- `"downstream_spillback"` — downstream occupancy too high
- `"min_green_window"` — ETA window too short

9. SUMO Simulator

Route: `/simulator`

Simulation Controls

Button	Action	API Call
▶ Start	Start SUMO simulation	<code>POST /api/simulation/start</code>
⏸ Pause	Pause simulation	<code>POST /api/simulation/pause</code>
▶ Resume	Resume simulation	<code>POST /api/simulation/resume</code>
■ Stop	Stop simulation	<code>POST /api/simulation/stop</code>
🔄 Reset	Reset all state	<code>POST /api/simulation/reset</code>

Speed Control

Use the **Speed Multiplier** slider or input to set simulation speed:

- `1×` — Real-time (1 simulation second = 1 wall-clock second)
- `10×` — 10× faster
- `100×` — Maximum speed

Scenario Selection

Before starting, select the scenario:

- **Baseline** — Normal traffic, no emergency
- **Emergency** — Emergency-ready scenario (normal demand)
- **Emergency Low** — Low traffic demand
- **Emergency High** — High traffic demand

Live Stats Panel

While running, shows:

- Current simulation time (sim seconds)
 - Step count
 - Vehicle count (total in network)
 - TraCI connection status
-

10. Scenario Builder

Route: </scenarios>

Running a Comparison

The Scenario Builder runs a **controlled experiment** measuring ITMS impact:

- **Configure Scenario:**
 - Emergency type (Ambulance / Fire Engine / Police)
 - Priority level
 - Origin junction
 - Destination junction
- **Click "Run Comparison":**
 - The system queues a background job
 - First runs **Baseline** (no ITMS)
 - Then runs **ITMS** (same conditions + full AI control)
 - Each run has identical traffic demand, same origin/destination, same timing
- **View Results:**
 - Delta Travel Time (s)
 - Delta Average Vehicle Delay (s)
 - Delta Queue Length
 - Delta Average Speed (m/s)
 - Delta Throughput (vehicles/hour)
- **Navigate to Analytics** to see historical comparison chart

Comparison Job States

State	Meaning
<code>queued</code>	Waiting to start
<code>running_baseline</code>	Executing baseline run
<code>running_itms</code>	Executing ITMS run
<code>completed</code>	Results available
<code>failed</code>	Error occurred

11. Analytics

Route: `/analytics`

What You See

- **Summary Cards:** Latest comparison delta values (e.g., "-80.6 s travel time saved")
- **Comparison Chart:** Bar chart of all completed comparisons by type + priority
- **Historical Table:** All completed `comparisons` records with timestamps

Chart Dimensions

- X-axis: Comparison job (type + origin → destination)
- Y-axis: Delta metric (select from dropdown: travel time / delay / queue / speed)
- Color: Emergency type (green = ambulance, red = fire, blue = police)

Note: Analytics data is only available after running at least one comparison via the Scenario Builder.

12. AI Decision Trace

Route: `/decisions`

What You See

A **live audit log** of every AI decision made by the system:

Event Type	Icon	Description
<code>route_computed</code>		New A* route computed (includes cost, edge count, ETA)
<code>route_switched</code>		Dynamic route change (reason, old ETA, new ETA)
<code>corridor_planned</code>		Green corridor planned for each junction
<code>safety_rejected</code>		Safety constraint blocked a corridor command
<code>signal_applied</code>		Signal state applied via TraCI
<code>signal_restored</code>		Normal signal program restored
<code>eta_updated</code>		ETA recalculated for a junction
<code>prediction_refresh</code>		ML prediction results updated

Filtering

- Filter by emergency ID
- Filter by event type
- Time range selector
- Export to JSON

Clicking a Decision

Click any decision row to open the **AI Copilot modal** pre-populated with a question about that decision. The LLM receives the full decision context.

13. System Settings

Route: `/settings`

Health Dashboard

Service	Indicator	Check
---------	-----------	-------

Backend API	 / 	<code>GET /health</code> HTTP probe
PostgreSQL	 / 	Backend internal DB pool probe
SUMO TraCI	 / 	Backend TraCI connection state
ML Prediction	 / 	<code>GET http://localhost:8100/health</code>
WebSocket	 / 	Frontend WS heartbeat

Configuration Display

Shows (read-only) the current active configuration values loaded from environment:

- Demo profile and city
- SUMO binary path
- Green corridor parameters
- Congestion thresholds

14. AI Copilot

Route: `/ai` (also accessible via floating `?` button anywhere)

Capabilities

The AI Copilot uses **Vultr Serverless Inference** (DeepSeek model by default) to answer natural language questions about the current traffic state.

Context injected automatically:

- Active emergencies (type, origin, destination, ETA, route)
- Active corridors (status, junctions, applied windows)
- Current traffic summary (congestion, average speed, queue)
- Predictions (next 30–120s per junction)
- Any selected decision event

Example Questions

- *"Why was the corridor for Ambulance #3 rejected at junction I4?"*
- *"What's the fastest route from W1 to E2 given current traffic?"*

- "How much time did ITMS save in the last comparison?"
- "Which junction is most congested right now?"

Requirements

`VULTR_SERVERLESS_INFERENCE_API_KEY` must be set in the backend `.env` to enable responses. Without it, the copilot shows an error message.

15. Roadside Devices

Route: `/roadside-devices`

What You See

Connected Roadside Priority Displays (CRPD) — IoT devices mounted at intersections that receive real-time corridor status.

Each device card shows:

- Device ID and location
- Connection status (ONLINE / OFFLINE)
- Current display mode (NORMAL / EMERGENCY / CORRIDOR)
- Last signal received timestamp
- Linked junction ID

Note: Roadside device integration requires physical CRPD hardware or a compatible simulator. In the current demo environment, this page shows the schema and UI without live device data.

16. Common Workflows

Workflow 1: Start a Simulation and Monitor Traffic

- Navigate to **Simulator** (`/simulator`)
- Select scenario: **Emergency**
- Click ► **Start**
- Navigate to **Command Center** (`/`)
- Observe: vehicles appear on the map, signal states update, congestion colors change

Workflow 2: Dispatch an Emergency Vehicle

- Ensure simulation is running (green status)
- Navigate to **Emergency Management** (`/emergencies`)
- Click **+ New Emergency**
- Select: Type = Ambulance, Priority = Critical, Origin = W1, Destination = E2
- Click **Dispatch**
- Return to **Command Center** — observe:
- Emergency vehicle appears on map with pulsing indicator
- Route highlighted in orange
- Green Corridor activates — junctions ahead flash green
- Corridor chain shows APPLIED → PASSED as vehicle advances

Workflow 3: Run Baseline vs ITMS Comparison

- Navigate to **Scenarios** (`/scenarios`)
- Configure: Ambulance, Critical, W1 → E2
- Click **Run Comparison** (takes ~60–120 seconds)
- Navigate to **Analytics** (`/analytics`)
- View delta: expected –80.6 s travel time saving

Workflow 4: Investigate a Safety Constraint Rejection

- Navigate to **AI Decision Trace** (`/decisions`)
- Filter by event type: `safety_rejected`
- Click the rejection event
- Read the reason (e.g., `"downstream_spillback"`)
- Click **Ask Copilot** — the AI explains the constraint in plain language

17. Status Indicators

Connection Status Banner

If the frontend cannot reach the backend WebSocket:

 **BACKEND DISCONNECTED** — Attempting to reconnect...

The banner shows reconnect attempt count and elapsed time.

Stale Data Banner

If traffic data is older than `TRAFFIC_STALE_AFTER_SECONDS` (default: 5 s):

 **STALE DATA** — Last update: X seconds ago

Simulation Status Chip

Chip Color	Meaning
 Blue	Stopped / Idle
 Green	Running
 Yellow	Paused
 Red	Error

18. Known Limitations

Limitation	Impact
Requires database to start	Most API calls fail without PostgreSQL; UI shows disconnected state
Requires SUMO	Simulation cannot start; all realtime data is unavailable
No persistent auth	Web app has no login; any user with network access can use it
Single active simulation	Cannot run two scenarios simultaneously
Comparison runs are sequential	~60–120 s wall-clock time per comparison at normal speed
City network requires build	<code>npm run fetch:city && npm run build:city-network</code> must run first
Screenshots show offline state	All screenshots in this documentation were taken without a live backend (database not running)

No mobile UI

Mobile app is a separate Android/iOS application (not covered here)
